

- Volume

This dataset is commonly referred to as OHLCV data.

Reliable historical data is essential because inaccurate or incomplete data can produce misleading results.

Strategy logic: The strategy logic specifies the rules utilised to produce trading signals. For instance, a moving average crossover method produces a buy signal when:

$$M A_{short} > M A_{long}$$

...and a sell signal when:

$$M A_{short} < M A_{long}$$

The backtesting model applies these rules successively to historical data.

Execution simulation: Execution simulation demonstrates how deals would have been completed in the market. Important factors considered are:

- Transaction expenses
- Slippage
- Bid-ask spread
- Market impact

Ignoring these factors may produce unrealistic performance results.

Portfolio management: Backtesting frameworks monitor portfolio values over time by managing:

- Cash balance
- Position sizes
- Asset holdings
- Profit and loss

Portfolio value at time t can be expressed as

$$Portfolio_t = Cash_t + Position_t \times Price_t$$

Performance evaluation: After executing a backtest, the framework computes numerous performance metrics, such as:

- Total return
- Sharpe ratio
- Maximum drawdown
- Win-loss ratio

- Volatility

The Sharpe ratio is defined as:

$$Sharpe = \frac{R_p - R_f}{\sigma_p}$$

...where:

- R_p is portfolio return
- R_f is risk-free return
- σ_p is portfolio return volatility

Why Python is suitable for backtesting ?

Strong data handling capabilities:

Python is widely regarded as the industry standard for time series data processing due to its specialised ecosystem of tools that automate everything from data ingestion to advanced forecasting. Pandas, Statsmodels, Darts, and NumPy are libraries that allow for efficient manipulation of financial time series data. Panda is the most useful of these, as it includes classes like Timestamp and DatetimeIndex for efficient temporal alignment and slicing. It has features for ingesting and processing, indexing, resampling, lagging-shifting, and window operations.

Visualisation tools: Traders can use visualisation libraries like Matplotlib and Plotly to analyse market movements, strategy performance, and drawdowns.

Integration with machine learning: Python frameworks like scikit-learn, TensorFlow, and PyTorch enable traders to incorporate machine learning models into their trading methods.

Large open source ecosystem: Many open source Python modules are specifically designed for quantitative finance and algorithmic trading.

Popular Python backtesting frameworks

Several Python frameworks have been created to backtest algorithmic trading techniques.

Backtrader: Backtrader is a popular Python-based framework for strategy backtesting and algorithmic trading.

Its comprehensive documentation and vibrant community support, which facilitate the efficient development of trading systems for both novice and experienced users, are among its main advantages. Complex trading methods, such as multi-data, multi-timeframe, and indicator-based approaches, can also be implemented on the platform. Furthermore, Backtrader facilitates a seamless transition from backtesting to real-time trading settings by offering integration possibilities with many brokers and live trading platforms.

Zipline: Zipline is an open source library for backtesting that was first created for quantitative studies. Event-driven backtesting, realistic trading simulation, integration with financial datasets, and the Pipeline API for factor-based models are some of its key features. Zipline is being extensively utilised in quantitative trading platforms and research settings. However, maintaining the framework is a limitation.

VectorBT: VectorBT is a backtesting library for high-performance vectorized backtesting. In contrast to conventional event-driven frameworks, VectorBT efficiently simulates techniques using NumPy and Pandas vectorization. It is incredibly fast, efficient for large datasets, allows interactive graphics, and most importantly, assists in optimising trading parameters. Because of these considerations, the vectorize method is effective for strategy research and optimisation.

PyAlgoTrade: PyAlgoTrade is another event-driven Python library for backtesting trading strategies. It supports technical indicators, event-driven architecture, performance measurements, and tools for strategy optimisation. Albeit less popular nowadays, this library is an effective pedagogical framework for understanding trading systems.

Lean Engine: QuantConnect has developed an advanced algorithmic trading framework called Lean Engine. Its features include cloud-based